

RELATÓRIO DE LABORATÓRIO

Mineração e análise dos 1.000 repositórios mais populares do GitHub

Lab01S03 — versão intermediária com RQ01–RQ08

Curso	Engenharia de Software
Disciplina	Laboratório de Experimentação de Software
Professor	Danilo Maia
Entrega	Lab01S03 — análise e visualização de dados
Integrantes	Pedro Henrique Maia Alves; Diogo C. Brunoro; [TERCEIRO INTEGRANTE — EDITAR]
Situação	Relatório intermediário; RQ09, RQ10 e snapshot final do board pendentes
Data de geração	25/08/2026

Belo Horizonte
2026

1. Introdução

O GitHub reúne projetos com diferentes níveis de maturidade, atividade e participação. Este laboratório investiga características dos 1.000 repositórios públicos mais estrelados retornados pela API GraphQL. Estrelas são usadas como definição operacional de popularidade, sem pressupor qualidade técnica.

Esta é a versão intermediária da sprint S03. Ela consolida os resultados atualizados das sete questões sugeridas pelo professor e introduz a RQ08, proposta pelo grupo para relacionar popularidade e intensidade anual de desenvolvimento. A aplicação Streamlit integra mineração, análise, visualização e exportação do CSV.

As hipóteses são informais e exploratórias: orientam a leitura descritiva, não foram pré-registradas e não substituem testes confirmatórios.

1.1 Questões de pesquisa e hipóteses informais

RQ	Pergunta	Hipótese informal
RQ01	Sistemas populares são maduros/antigos?	H01: a idade mediana é superior a cinco anos.
RQ02	Sistemas populares recebem muita contribuição externa?	H02: a mediana supera 500 PRs mescladas e a distribuição é assimétrica.
RQ03	Sistemas populares lançam releases com frequência?	H03: a maioria possui releases e a mediana acumulada supera 20.
RQ04	Sistemas populares são atualizados com frequência?	H04: a maioria recebeu push nos últimos 30 dias.
RQ05	Sistemas populares usam linguagens populares?	H05: mais da metade usa uma linguagem do Top 10 Octoverse 2025.
RQ06	Sistemas populares possuem alto percentual de issues fechadas?	H06: a mediana de fechamento supera 80% entre projetos com issues.
RQ07	Linguagens populares recebem mais contribuição, releases e atualização?	H07: o grupo Top 10 tem maiores medianas de PRs e releases e menor recência.
RQ08	Popularidade está associada à intensidade anual de desenvolvimento?	H08: estrelas apresentam associação positiva ao menos moderada com PRs/ano e releases/ano.

2. Desenvolvimento

2.1 Evolução do trabalho em S01, S02 e S03

Na S01 foram implementadas a consulta GraphQL inicial, a transformação tabular, as RQ01–RQ07 e a primeira interface Streamlit. Na S02, o coletor em lote passou a percorrer cursores até reunir exatamente 1.000 nomes únicos, exportar CSV e registrar o primeiro snapshot do Project.

Na S03 foi implementada a Issue #44 (RQ08), com duas taxas anualizadas, correlações de Spearman, quartis de estrelas, diagnóstico de outliers e visualizações no Streamlit. O fluxo da RQ08 está integrado ao main.py, ao CSV canônico e ao relatório. RQ09, RQ10 e o print final do board permanecem explicitamente pendentes para a conclusão da sprint.

Sprint	Atividade	Responsável	Rastreabilidade
S01	Coleta inicial, RQ01–RQ07 e Streamlit	Pedro e Diogo	Issues e PRs da S01
S02	Paginação para 1.000, CSV, validação e hipóteses	Pedro e Diogo	Issues #8–#15 e #22–#38
S03	RQ08 — popularidade e intensidade	[TERCEIRO INTEGRANTE — EDITAR]	Issue #44
S03	RQ09 — colaboração e releases	Pendente	Issue #45
S03	RQ10 — maturidade e issues	Pendente	Issue #46
S03	Evolução de exibição e busca	Pendente	Issue #47

2.2 Arquitetura e fluxo implementado

- main.py pagina a consulta GraphQL, valida avanço dos cursores e encerra apenas com 1.000 repositórios únicos;
- utils/dataframe.py converte o snapshot bruto e calcula métricas derivadas reproduzíveis;
- analysis contém módulos independentes de RQ01 a RQ08 e exporta JSON estrito;
- app.py apresenta uma aba por RQ e permite baixar o CSV com as taxas da RQ08;
- gerar_relatorio_docx.py recalcula todos os números exibidos a partir do mesmo CSV canônico.

2.3 Inovação: aplicação Streamlit

A inovação do grupo é uma aplicação interativa em Streamlit. A versão integrada à RQ08 mostra os coeficientes, o tamanho das amostras, os quartis de estrelas, a sensibilidade e dois gráficos de dispersão. Os tooltips preservam valores originais; estrelas usam escala logarítmica e as taxas usam symlog, mantendo observações iguais a zero.

Carregamento automático de snapshots e a unificação completa dos fluxos de busca foram mantidos fora desta atividade para não absorver o escopo da Issue #47.

3. Metodologia

3.1 Seleção e paginação

A população operacional corresponde aos repositórios públicos encontrados por `stars:>0 is:public sort:stars-desc`. A conexão `search` é percorrida em lotes de até 50. Cada nova chamada recebe em `after` o `endCursor` anterior; `hasNextPage`, `cursor` ausente ou repetido e página vazia são verificados. Duplicatas por `nameWithOwner` são descartadas até atingir exatamente 1.000 projetos.

3.2 Transformação e métricas

RQ	Métrica operacional
RQ01	(data da transformação – <code>createdAt</code>) / 365,25, em anos
RQ02	<code>pullRequests(states: MERGED).totalCount</code> ; não identifica autoria externa
RQ03	<code>releases.totalCount</code> acumulado
RQ04	dias entre a transformação e <code>pushedAt</code>
RQ05	<code>primaryLanguage</code> comparada ao Top 10 Octoverse 2025
RQ06	100 × issues fechadas / (abertas + fechadas)
RQ07	medianas de PRs, releases e recência por grupo de linguagem
RQ08	PRs/ano e releases/ano, divididos pela idade registrada no CSV

A idade é arredondada para duas casas antes da anualização; as duas taxas são arredondadas para quatro casas. Idades não positivas produzem `null`, nunca `NaN` ou infinito.

3.3 Procedimento analítico da RQ08

A análise principal inclui idade ≥ 1 ano ($n = 917$); a sensibilidade inclui toda idade positiva ($n = 1000$). Spearman é calculado como Pearson entre postos médios, sem SciPy. Os quartis são formados após ordenação estável por estrelas e nome. Outliers seguem $1,5 \times \text{IQR}$ e permanecem em todas as análises.

A anualização reduz o efeito mecânico do tempo de existência, mas não controla domínio, tamanho da equipe, governança, automação ou trajetória histórica. As associações não permitem inferência causal.

4. Resultados

4.1 Validação do snapshot

O snapshot contém 1.000 linhas, 1.000 nomes e 1.000 URLs únicas. As estrelas estão em ordem não crescente. O CSV contém as taxas PRs por ano e Releases por ano, e os oito arquivos JSON registram total_repositorios = 1.000.

4.2 Visualizações e resultados por RQ

4.2.1 RQ01 — Sistemas populares são maduros/antigos?

A idade mediana foi 7,71 anos e a média 7,64. O intervalo observado foi de 0,03 a 18,37 anos. As faixas <2, 2–<5, 5–<10 e ≥10 anos reuniram, respectivamente, 141, 185, 331, 343 projetos.

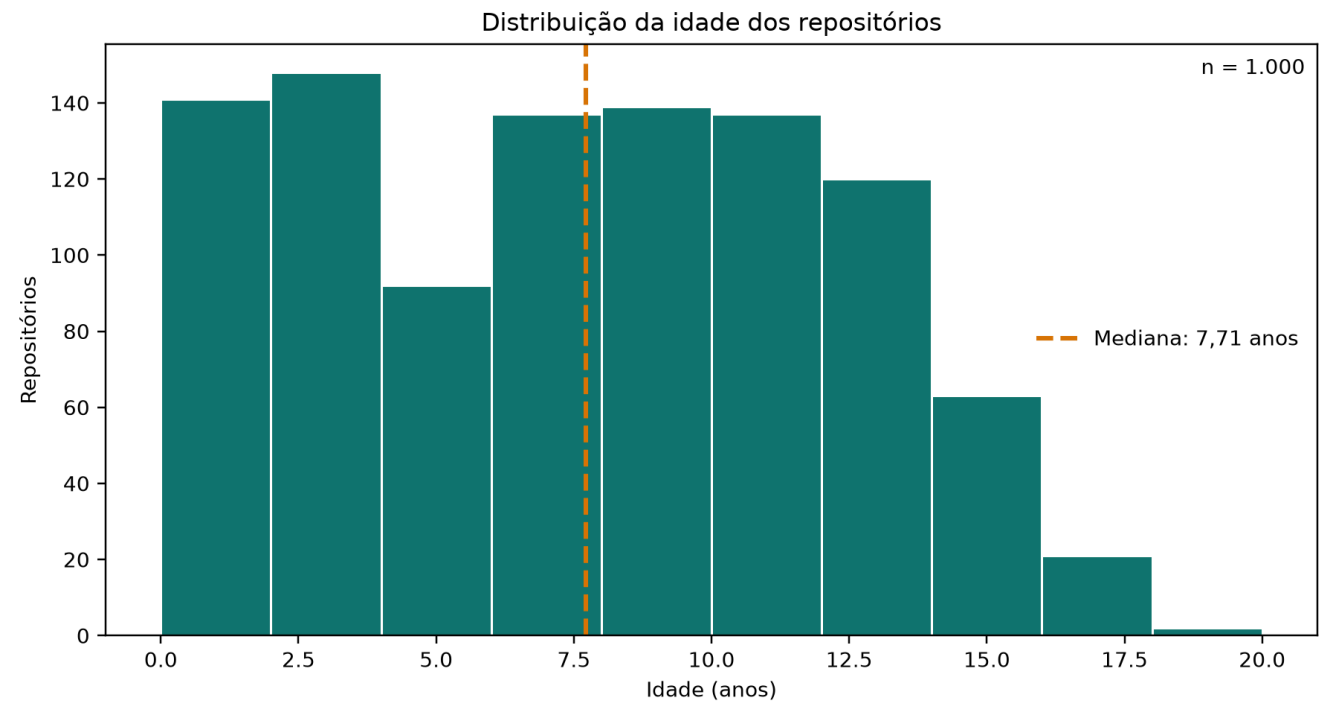


Figura 2 — Distribuição da idade dos repositórios.

Fonte: elaborado pelo grupo com dados da API GraphQL do GitHub, 2026.

4.2.2 RQ02 — Sistemas populares recebem muita contribuição externa?

A mediana foi 768 PRs mescladas, a média 4.255,21, Q1 174,00, Q3 3.409,00 e o máximo 103.641. Há 20 projetos com zero e 124 outliers superiores.

Repositório	PRs mescladas
firstcontributions/first-contributions	103.641
llvm/llvm-project	98.120
elastic/elasticsearch	95.877

PRs mescladas aproximam volume de contribuição, mas a consulta não permite afirmar que a contribuição veio de pessoas externas ao projeto.

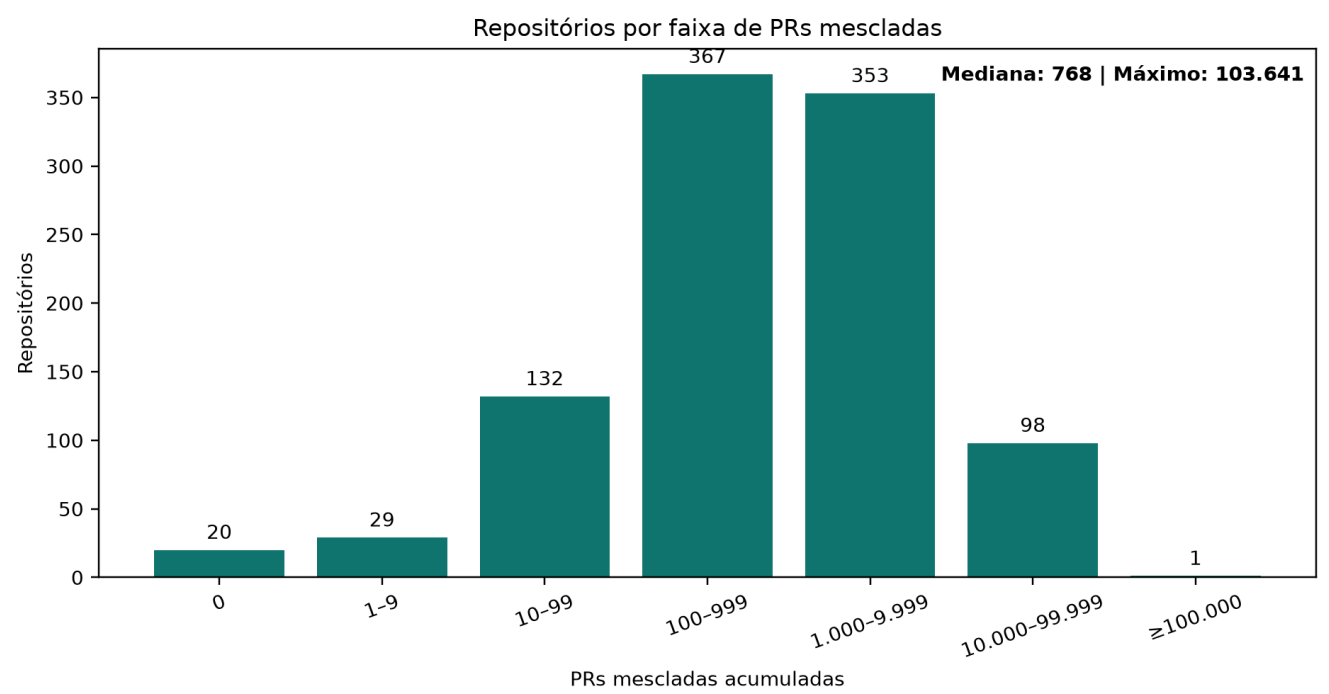


Figura 3 — Distribuição por faixa de pull requests mescladas.

Fonte: elaborado pelo grupo com dados da API GraphQL do GitHub, 2026.

4.2.3 RQ03 — Sistemas populares lançam releases com frequência?

A mediana acumulada foi 42, a média 158,77, Q1 0,00, Q3 153,00 e o máximo 6.951. 728 projetos (72,8%) possuem ao menos uma release; 272 possuem zero. Foram detectados 92 outliers superiores.

Repositório	Releases
ggml-org/llama.cpp	6.951
gradio-app/gradio	5.095
vercel/next.js	3.823

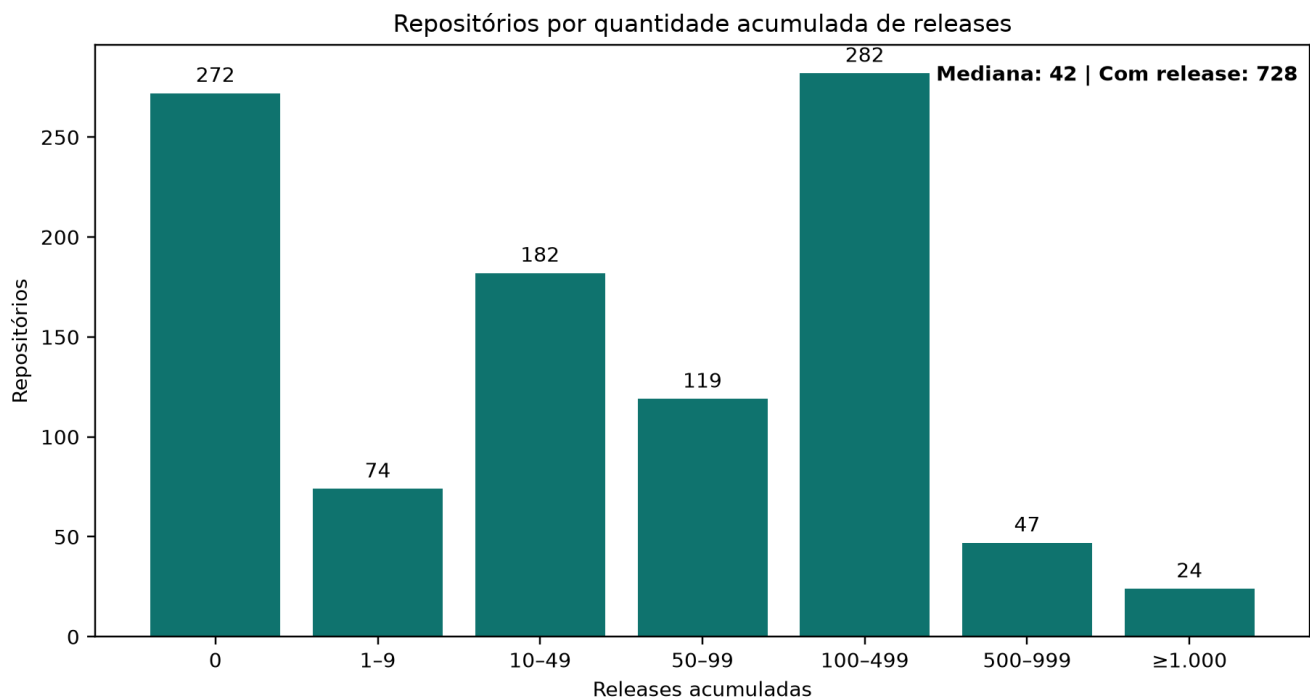


Figura 4 — Distribuição por quantidade acumulada de releases.

Fonte: elaborado pelo grupo com dados da API GraphQL do GitHub, 2026.

4.2.4 RQ04 — Sistemas populares são atualizados com frequência?

A mediana desde o último push foi 1,68 dias, a média 113,34 e o máximo 2.457,87. 728 projetos (72,8%) estavam em até 30 dias; 119 estavam há mais de um ano. O critério IQR identificou 198 outliers superiores.

Recência	Repositórios
Até 1 dia	436
>1 a 7 dias	177
>7 a 30 dias	115
>30 a 365 dias	153
>365 dias	119

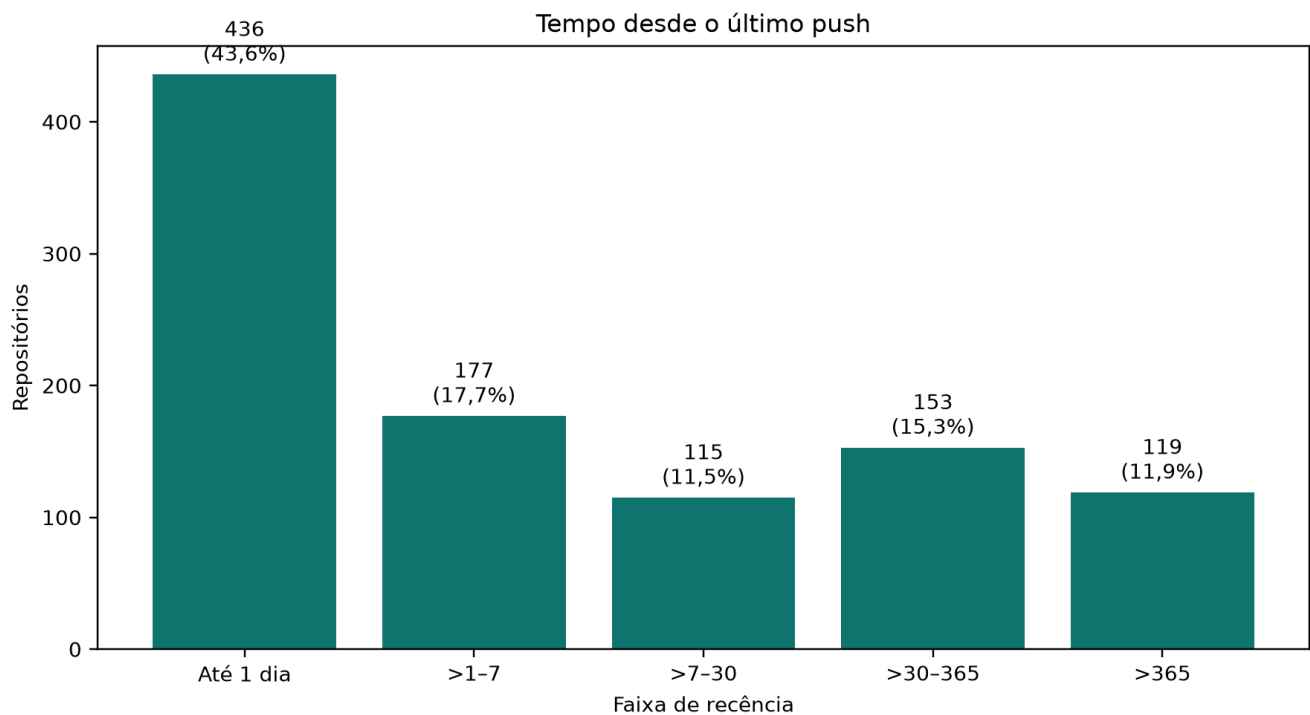


Figura 5 — Distribuição do tempo desde o último push.

Fonte: elaborado pelo grupo com dados da API GraphQL do GitHub, 2026.

4.2.5 RQ05 — Sistemas populares usam linguagens populares?

Foram observadas 43 linguagens definidas. O Top 10 Octoverse aparece em 703 projetos (70,3%) e em 77,0% daqueles com linguagem definida. 87 projetos não possuem linguagem primária definida.

Linguagem	Repositórios
Python	228
TypeScript	173
JavaScript	110
Não definida	87
Go	77
Rust	57
C++	42
Java	41

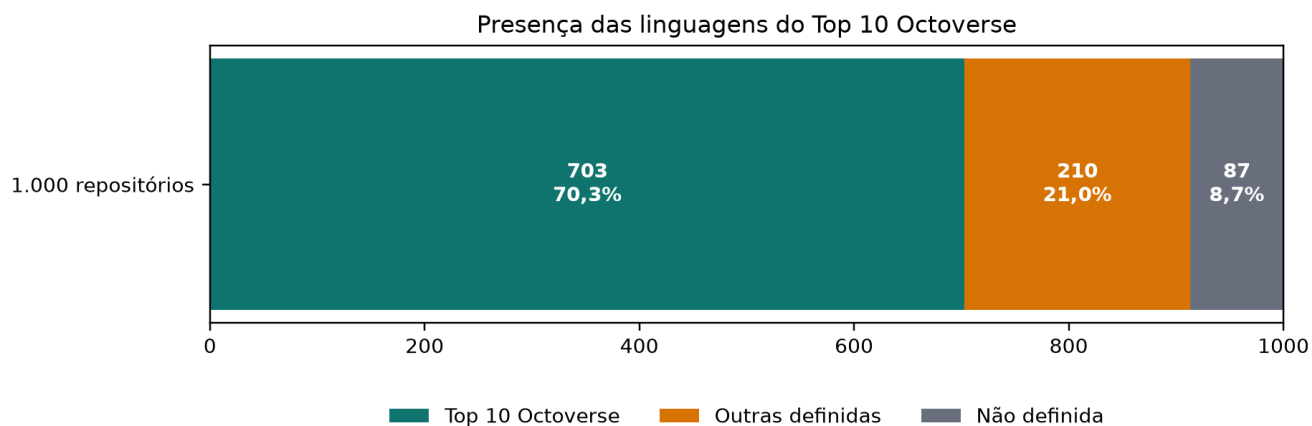


Figura 6 — Participação das linguagens do Top 10 GitHub Octoverse 2025.

Fonte: elaborado pelo grupo com dados da API GraphQL do GitHub, 2026.

4.2.6 RQ06 — Sistemas populares possuem alto percentual de issues fechadas?

A taxa foi calculável em 957 projetos; 43 não possuíam issues. Entre os válidos, a mediana foi 87,60%, a média 80,35%, Q1 70,90%, Q3 96,80%, mínimo 7,50% e máximo 100,00%. 618 (64,6%) alcançaram pelo menos 80%.

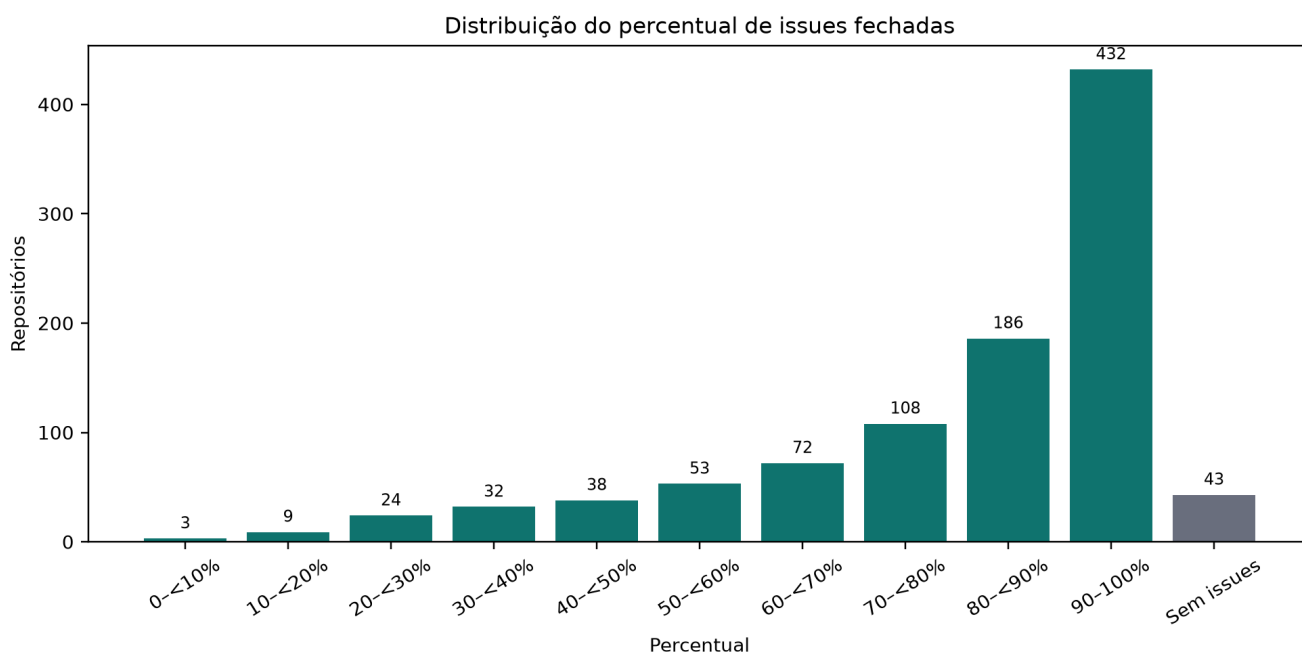


Figura 7 — Distribuição do percentual de issues fechadas.

Fonte: elaborado pelo grupo com dados da API GraphQL do GitHub, 2026.

4.2.7 RQ07 — Linguagens populares recebem mais contribuição, releases e atualizações?

A comparação exclui 87 projetos sem linguagem definida. As estatísticas abaixo foram recalculadas do CSV para os mesmos grupos do Octoverse.

Grupo	N	Mediana PRs	Mediana releases	Mediana dias
Top 10 Octoverse	703	1.000	64	1,14
Outras definidas	210	660	30	2,29

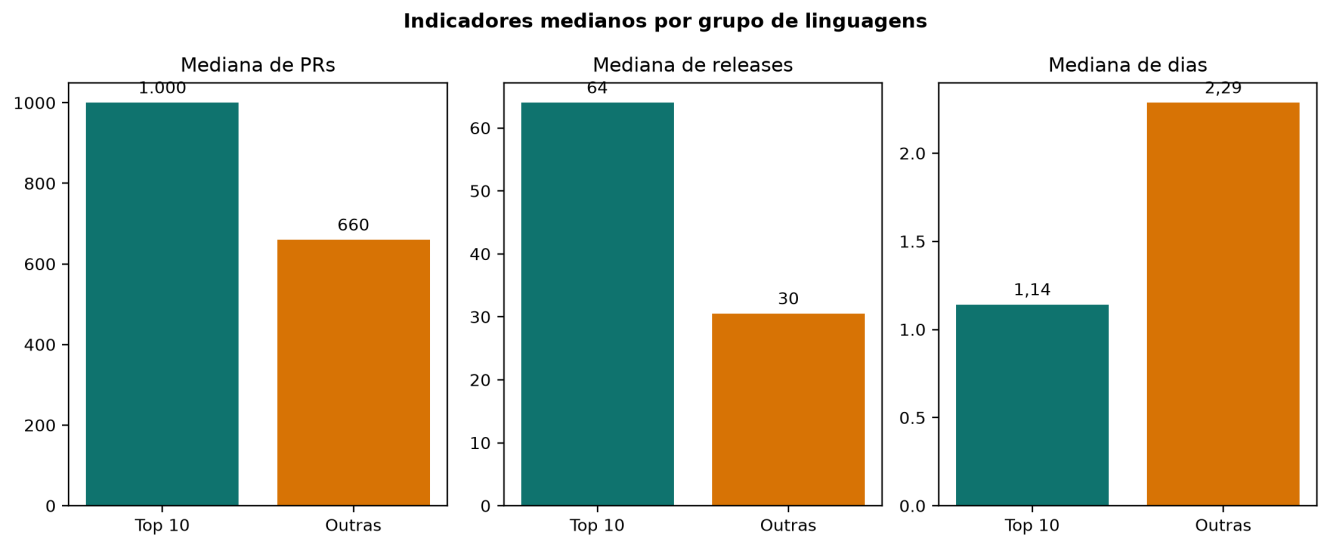


Figura 8 — Indicadores medianos por grupo de linguagens.

Fonte: elaborado pelo grupo com dados da API GraphQL do GitHub, 2026.

4.2.8 RQ08 — Popularidade e intensidade anual de desenvolvimento

Na amostra principal (idade ≥ 1 ano; $n = 917$), estrelas $\times$ PRs/ano apresentou $\rho = 0,1534$ e estrelas $\times$ releases/ano $\rho = 0,0066$. Há 83 projetos com menos de um ano; nenhum outlier foi removido.

Quartil de estrelas	N	Limites de estrelas	Mediana PRs/ano	Mediana releases/ano
Q1	230	33.022–38.502	79,1039	5,3305
Q2	229	38.550–48.469	110,9813	7,4866
Q3	229	48.485–72.000	148,7047	7,1702
Q4	229	72.148–542.947	192,7794	4,8193

Na sensibilidade com todas as idades positivas ($n = 1000$), os coeficientes foram $\rho = 0,1451$ para PRs/ano e $\rho = 0,0070$ para releases/ano. O IQR marcou 112 extremos em PRs/ano, 92 em releases/ano e 165 na união.

RQ08 — Popularidade e intensidade anual (idade ≥ 1 ano)

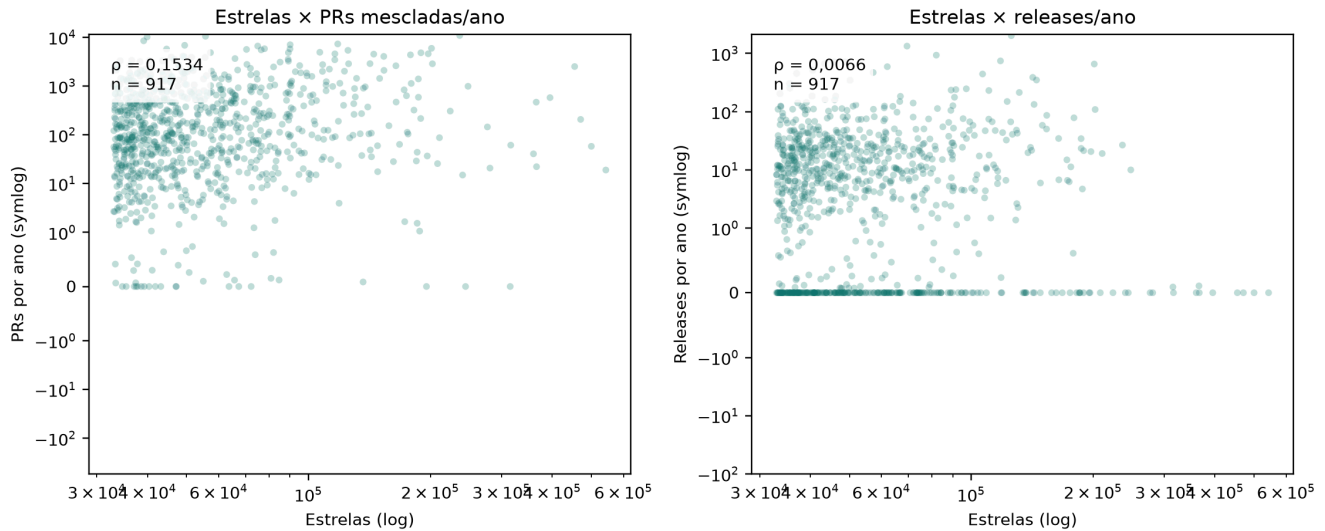


Figura 9 — Dispersões de estrelas versus PRs/ano e releases/ano.

Fonte: elaborado pelo grupo com dados da API GraphQL do GitHub, 2026.

4.3 Discussão e avaliação das hipóteses

4.3.1 RQ01

A mediana de 7,71 anos superou cinco anos. Projetos antigos tiveram mais tempo para acumular estrelas; o resultado não estabelece causalidade.

4.3.2 RQ02

A mediana de 768 superou 500. A diferença entre média e mediana e os 124 extremos mostram assimetria. A autoria externa não é identificável.

4.3.3 RQ03

72,8% possui release e a mediana é 42. O acumulado não mede intervalos nem frequência temporal.

4.3.4 RQ04

72,8% estava em até 30 dias. O último push mede recência, não frequência histórica, e pode refletir automação.

4.3.5 RQ05

O Top 10 representa 70,3% da amostra. O resultado depende do ranking temporal e primaryLanguage simplifica projetos multilíngues.

4.3.6 RQ06

A mediana de 87,60% descreve somente projetos com issues. A taxa não informa tempo, complexidade ou qualidade do fechamento.

4.3.7 RQ07

A comparação é descritiva. Diferenças entre grupos podem refletir idade, domínio, comunidade, governança e tamanhos desiguais, não o efeito da linguagem isoladamente.

4.3.8 RQ08

A relação com PRs/ano foi positiva fraca ($p = 0,1534$) e com releases/ano foi positiva desprezível ($p = 0,0066$). Assim, H08 não é sustentada no critério de duas associações positivas ao menos moderadas. A sensibilidade manteve a mesma interpretação geral.

Hipótese	Avaliação exploratória
H01	Compatível
H02	Compatível descritivamente
H03	Compatível para presença/acumulado
H04	Compatível para recência
H05	Compatível
H06	Compatível entre projetos com issues
H07	Compatível descritivamente
H08	Não sustentada: as duas associações não foram ao menos moderadas

4.3.9 Ameaças à validade

Validade de construto. Estrelas aproximam popularidade; PRs mescladas não garantem contribuição externa; releases acumuladas e último push não medem frequência histórica; linguagem primária não descreve todo o código.

Anualização. Dividir acumulados pela idade supõe uma taxa média constante e amplifica valores de projetos muito jovens. Por isso, a análise principal exige um ano e a sensibilidade inclui todas as idades positivas.

Outliers. Projetos com automação ou fluxos incomuns produzem taxas extremas. Eles foram identificados por $1,5 \times$ IQR e mantidos; a correlação por postos reduz, mas não elimina, sua influência interpretativa.

Causalidade e confundimento. O estudo observacional não controla domínio, idade, organização, tamanho da equipe ou governança. Associação entre estrelas e intensidade não demonstra que uma variável cause a outra.

Validade externa e temporal. A amostra cobre somente os 1.000 repositórios públicos mais estrelados no instante da coleta. Resultados não se generalizam automaticamente e podem variar entre snapshots.

5. Conclusão

A coleta oficial reuniu 1.000 repositórios únicos, ordenados por estrelas, e gerou CSV e oito JSONs coerentes. Todos os valores do relatório foram recalculados do mesmo snapshot, reduzindo o risco de divergência entre dados, texto, tabelas e gráficos.

RQ01–RQ07 descrevem maturidade, contribuições, releases, atualização, linguagens e issues. A RQ08 acrescenta controle simples por idade: na amostra principal, p foi 0,1534 para PRs/ano e 0,0066 para releases/ano. Os resultados não sustentam a hipótese de duas associações positivas ao menos moderadas e não autorizam conclusão causal.

O Streamlit agora comunica a RQ08 junto às questões anteriores e oferece download das novas taxas. Esta versão é intermediária: RQ09, RQ10, revisão final do texto e anexo com o print do board e a política de WIP ainda devem ser concluídos antes do Relatório Final.

6. Referências

GITHUB. GraphQL API documentation. <https://docs.github.com/en/graphql>. Acesso em: 25 ago. 2026.

GITHUB. Using pagination in the GraphQL API. <https://docs.github.com/en/graphql/guides/using-pagination-in-the-graphql-api>. Acesso em: 25 ago. 2026.

GITHUB. Octoverse 2025. <https://github.blog/news-insights/octoverse/>. Acesso em: 25 ago. 2026.

KALLIAMVAKOU, E. et al. The promises and perils of mining GitHub. MSR, 2014. <https://doi.org/10.1145/2597073.2597074>.

PANDAS. Pandas documentation. <https://pandas.pydata.org/docs/>.

STREAMLIT. Streamlit documentation. <https://docs.streamlit.io/>.

VEGA-ALTAIR. Declarative visualization in Python. <https://altair-viz.github.io/>.

Anexo A — Fluxo do GitHub Project

PENDÊNCIA DA VERSÃO INTERMEDIÁRIA: inserir no Relatório Final o print do board mostrando o fluxo completo de S01 a S03 e a política de WIP em uso. O primeiro snapshot da S02 permanece preservado no histórico do projeto.

INSERIR SNAPSHOT FINAL DO BOARD E EVIDÊNCIA DA POLÍTICA DE WIP